CrewAI

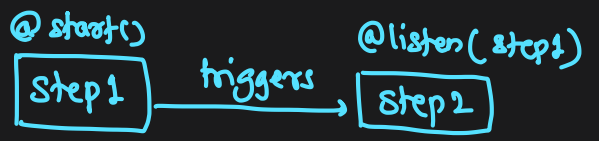
multi-Agent apps

Introduction

- CrewAI is an open-source Python framework for orchestrating autonomous AI agents and building complex, production-ready workflows
- Its core design philosophy balances **autonomy** with **precise control**: it provides the collaborative intelligence of role-playing agent teams, while also offering the structured, stateful orchestration needed for reliable production systems
- The framework is built around two complementary abstractions
  - **Crews**
  - **Flows**

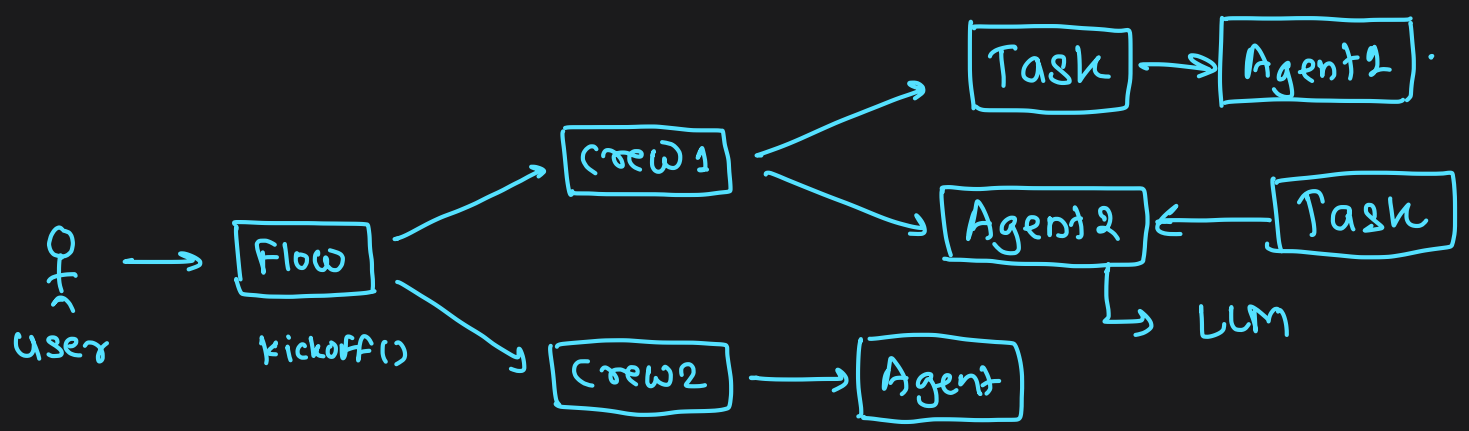
Flows – The Backbone

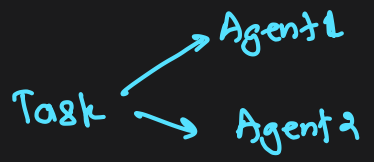
- A **Flow** is the structural foundation of a CrewAI application
- Think of it as the "manager" or "process definition" that defines the steps, logic, and data movement through your system → workflow
- Flows provide three critical capabilities:
  - **State Management**
    - Flows persist data across steps and executions, allowing you to pass information between different Crews, maintain context, and handle user inputs
    - State can be either unstructured (a flexible dictionary) or structured (using Pydantic models for type safety and validation)
  - **Event-Driven Execution**
    - Flows trigger actions based on events or external inputs
    - The @start() decorator marks entry points, while @listen() decorators define steps that execute in response to other steps completing
  - **Control Flow**
    - Flows support conditional logic, loops, and branching—essential for handling edge cases and ensuring predictable application behavior



Crews: The Intelligence

- A **Crew** is a team of autonomous agents that collaborate to solve a specific task delegated by a Flow
- Crews are the "units of work" that do the heavy lifting requiring creativity, exploration, and collaboration
- Each Crew consists of **Agents** (specialized team members with defined roles, goals, and tools) and **Tasks** (discrete work units)
- The Crew defines how these agents collaborate through its **Process** (Sequential or Hierarchical)



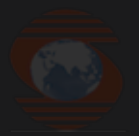


How They Work Together



- The interaction pattern is straightforward
  - a Flow starts and manages state, then delegates a complex task to a Crew
  - The Crew's agents collaborate autonomously to complete the task, returning the result to the Flow, which continues execution based on that result
- **The recommended production pattern is "Flow-first"**
- Even if you only need a single Crew, wrapping it in a Flow provides state management, observability, and the structure needed for robust deployment

Agents: The Team Members



- It is a specialized AI team member defined by three required attributes: **role**, **goal**, and **backstory**
- The **role** defines what the agent is (e.g., "Senior Data Researcher") → system prompt
- The **goal** defines what the agent aims to achieve → deliverable
- The **backstory** provides situational context that shapes the agent's behavior and expertise → context
- **Key Agent Parameters**
  - **LLM configuration:** The llm parameter determines which language model the agent uses (default: OpenAI's GPT-4o-mini). You can pass a model name string (e.g., 'gpt-4', 'claude-2') or an LLM class instance for advanced configuration
  - **Memory:** Setting memory=True enables the agent to maintain conversation history and context across multiple interactions, which is useful for complex, multi-step tasks → short term memory
  - **Context window management:** The respect_context_window parameter (default True) automatically summarizes content when the conversation exceeds the model's token limit. Setting it to False causes execution to stop with an error instead, preserving all context for cases where information loss is unacceptable
  - **Tools:** Agents are equipped with tools to interact with external systems. Tools can be specified at the agent level, and task-level tool specifications can override the agent's defaults for fine-grained control

Tasks: The Work Units

- A **Task** represents a discrete unit of work
- Each task has a **description** (what to do), an **expected_output** (what the result should look like), and an assigned **agent**
- **Task Configuration**



  - **Dependencies:** Tasks can declare dependencies on other tasks, forming a structured execution order. The output of a preceding task can serve as context for subsequent tasks
  - **Output schemas:** Tasks support output_pydantic and output_json parameters for structured outputs. This is critical for production applications: structured outputs prevent parsing errors and ensure type safety when passing data between tasks or to your application
  - **Guardrails:** Task Guardrails allow you to validate task outputs before they are accepted. A guardrail function returns a tuple of (bool, Any)—if the first element is False, the second element provides feedback for the agent to retry
  - **Callbacks:** task_callback executes after each task completes; step_callback executes after each step in an agent's execution

Processes: How Crews Collaborate

- The **Process** defines the execution strategy for a Crew's tasks
- CrewAI supports two process types, specified via the Process enum
- **Sequential Process (Default)** → deterministic
  - Tasks execute in the exact order they appear in the task list
  - The output of one task serves as context for the next
  - This is ideal for linear workflows where each step builds on the previous one
  - **Key characteristics:** Simple, easy to monitor, and requires that every task has an assigned agent
- **Hierarchical Process** → supervisor → orchestration
  - **A manager agent coordinates the workflow:** it delegates tasks to appropriate crew members based on their roles, validates outcomes, and ensures quality. This simulates a traditional organizational hierarchy
  - **The manager agent** can be created automatically by CrewAI or defined explicitly with custom attributes
  - A custom manager gives you control over the manager's role, goal, backstory, and capabilities
  - **Workflow:** The manager assigns tasks strategically, agents execute with oversight, and the manager validates results before proceeding. Despite the hierarchical structure, tasks still follow a logical order

Tools: Extending Agent Capabilities

- Tools allow agents to interact with external systems—APIs, databases, search engines, file systems
- **Creating Custom Tools**
  - There are two primary approaches for custom tools
  - **Subclassing BaseTool:** Define a Pydantic input schema (args_schema), set the tool's name and description, and implement the _run method
  - **Using the @tool decorator:** For simpler tools, decorate a function with @tool("Tool Name"). The function's docstring serves as the description
- **Typed Outputs (Best Practice)**
  - When a tool returns structured data, define a Pydantic output model
  - This allows the agent to read the result as clear fields rather than guessing from plain text.
  - CrewAI infers the output schema from the Pydantic return annotation on a BaseTool subclass, or you can set result_schema explicitly with the decorator
  - Typed outputs are essential for results with stable fields like IDs, statuses, scores, or lists. They eliminate parsing ambiguity and improve agent reliability
- **Tool Caching and Async**
  - Custom caching strategies can be defined via cache_function
  - Async tools are supported by using async def with the @tool decorator, or by implementing both _run and _arun methods in a BaseTool subclass

Summary

Concept	Purpose	Key Configuration
Flow	Application backbone; state and control	Flow[StateModel], @start, @listen, @persist
Crew	Team of agents executing a task	agents, tasks, process, manager_llm
Agent	Specialized team member	role, goal, backstory, llm, tools, memory
Task	Discrete work unit	description, expected_output, agent, context, guardrails
Process	Collaboration strategy	Sequential (default) or Hierarchical
Tool	External system interface	BaseTool subclass or @tool decorator
LLM	Language model connection	Native SDK (OpenAI/Anthropic/etc.) or LiteLLM fallback